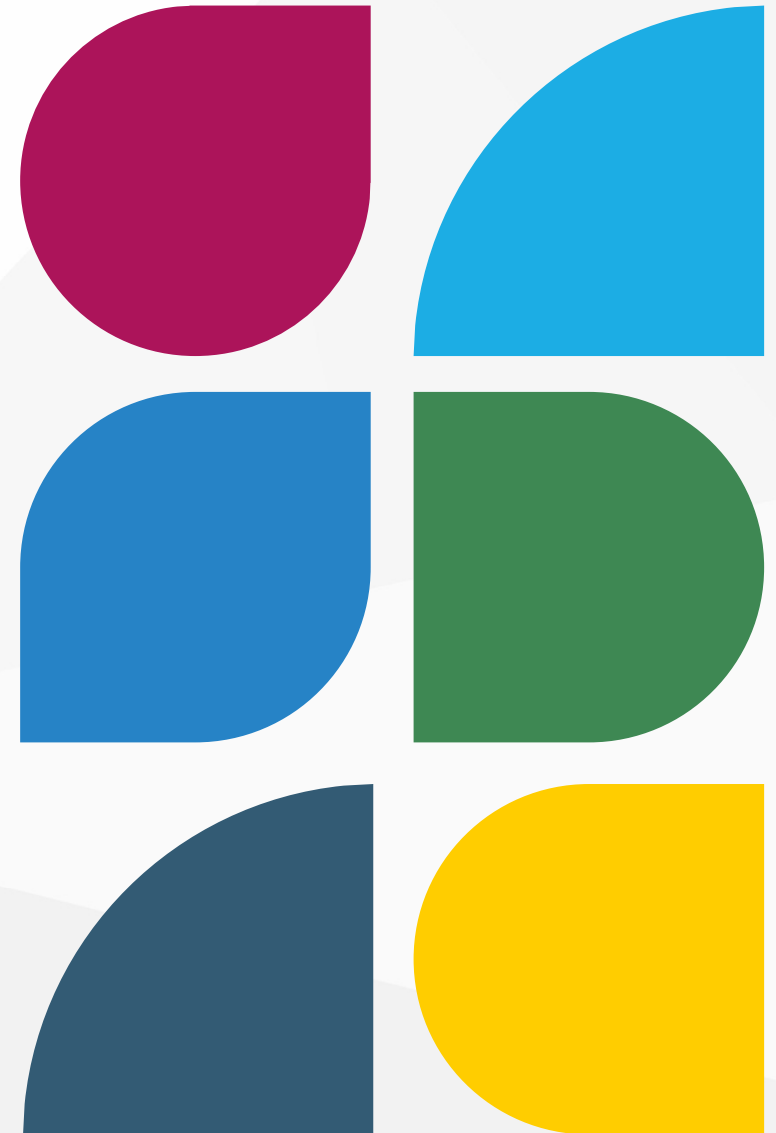




GESTION DE PROJETS JAVA

MavenTM

Concepteur Développeur d'Applications



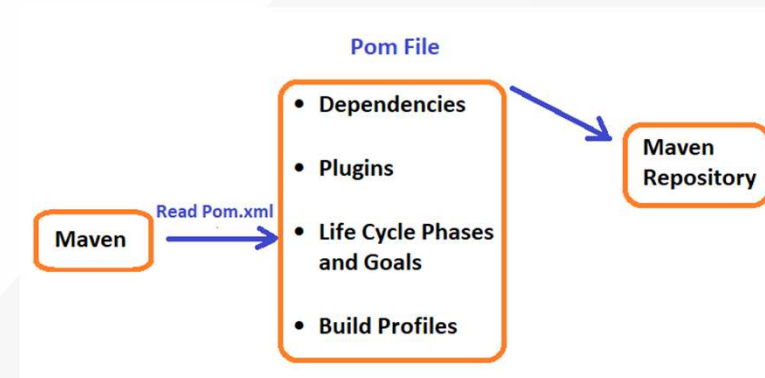
MAVEN

Maven est un **outil de gestion de projet logiciel** permettant la gestion de l'installation des dépendances et automatiser sa construction (compilation, test, production de livrable...)

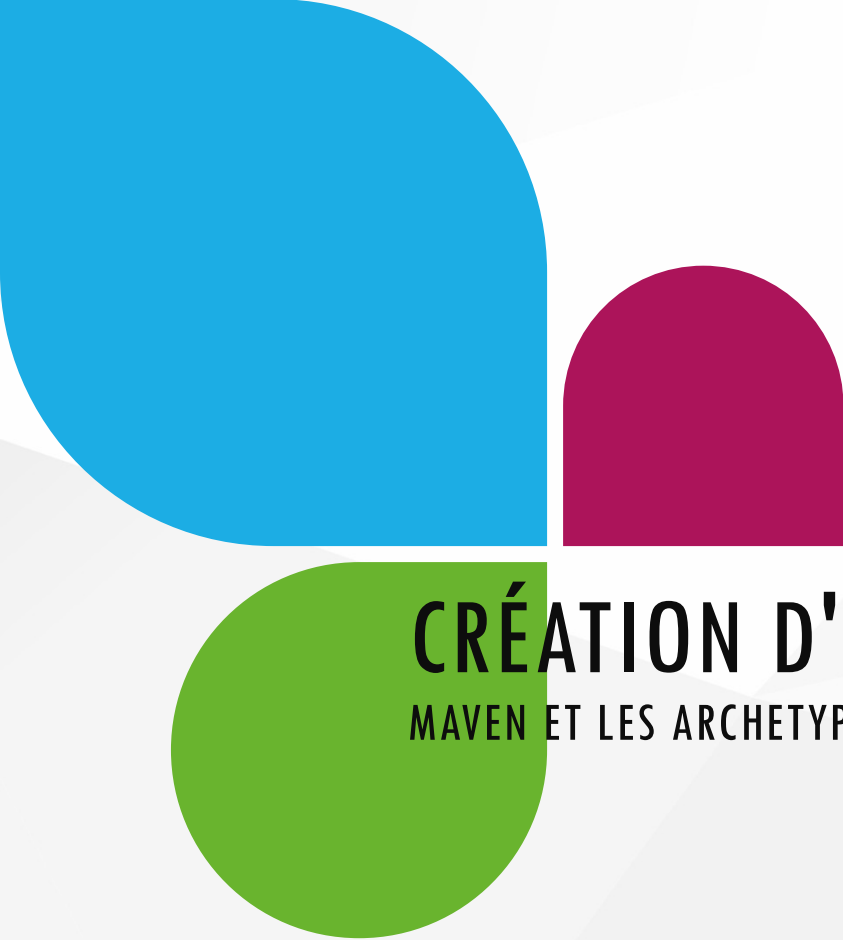
Maven est **open source**, développé en Java et maintenu par l'Apache Software Foundation

Maven peut également être utilisé pour construire et gérer des projets écrits avec des langages tels que C#, ruby et d'autres langages de programmation.

- Il est basé sur le concept du modèle d'objet de projet (**POM**).
- Son architecture est présentée ci-dessous :



- **Maven permet au développeur :**
 - d'automatiser le traitement de la création du format de dossier original,
 - la réalisation de l'assortiment et des tests,
 - l'emballage et le déploiement du résultat final.
- Il réduit le nombre considérable d'étapes du processus de base et permet de réaliser une compilation en une seule étape.



CRÉATION D'UN PROJET

MAVEN ET LES ARCHETYPES

INSTALLATION DE MAVEN

Site officiel de maven :

<https://maven.apache.org/>

Téléchargement :

<https://maven.apache.org/download.cgi>



L'installation de Maven s'effectue en téléchargeant et décompressant le logiciel dans un des dossiers de votre ordinateur. Simple et efficace...

- Mais avec IntelliJ, le logiciel est déjà installé donc ce téléchargement n'est pas forcément nécessaire et vous pouvez très bien utiliser cette version en mode console. Dans ce cas, passez à la diaporama 6.*

Bien qu'aujourd'hui, beaucoup d'IDE intègre Maven, il est bon de connaître son installation et son utilisation directement en ligne de commande.

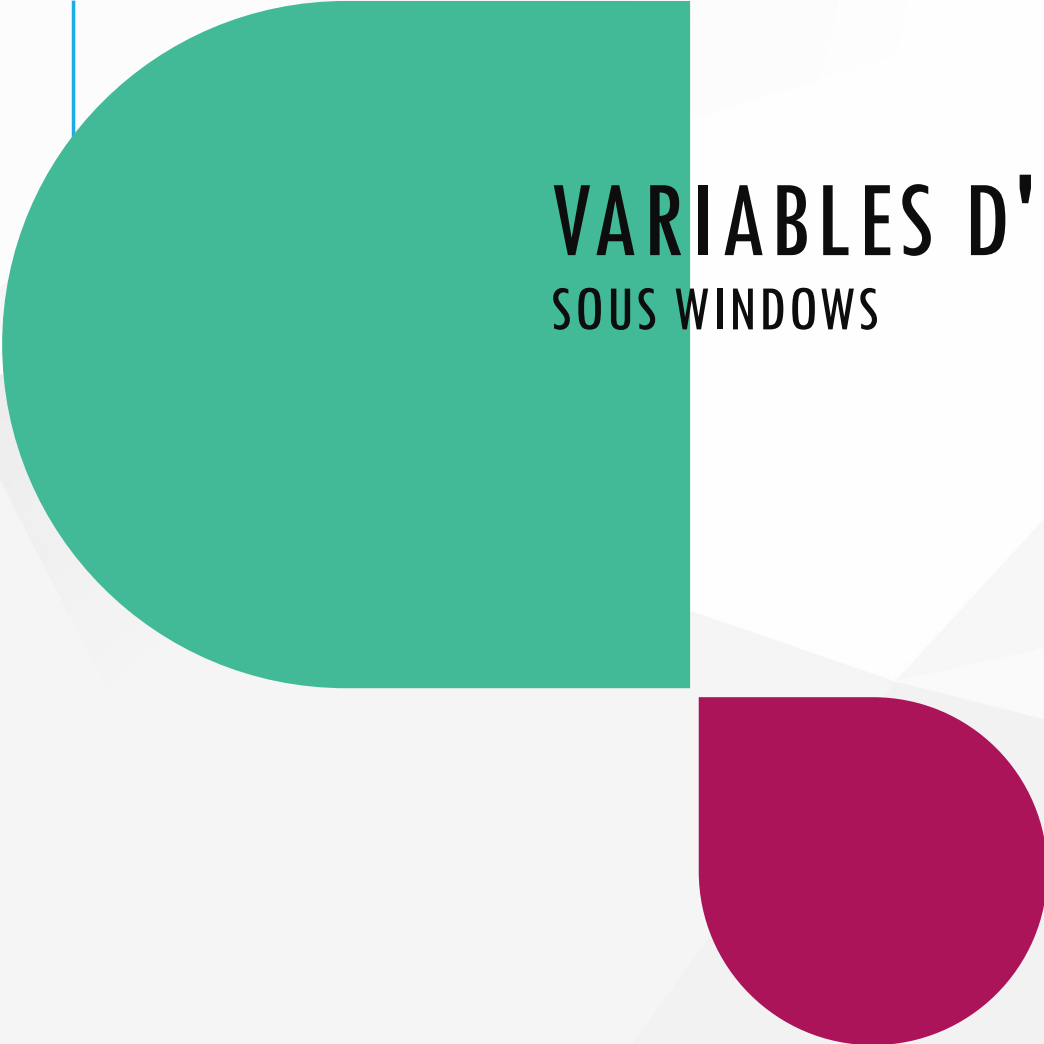
1. Après téléchargement, décompresser le fichier et placer le dans un répertoire de travail.
2. Ajouter dans la variable d'environnement PATH, le chemin vers le dossier bin de Maven.
3. Tester l'installation, depuis n'importe où avec la commande `mvn -v` qui donnera la version installée.

CONFIGURATION DE MAVEN

La configuration se fait dans le fichier `.m2/setting.xml` dans le dossier utilisateur (sous Windows).

1. Ce fichier n'existe pas par défaut, copier celui contenu dans le répertoire `conf` de l'installation de Maven.
2. Maven va télécharger les dépendances dans un repository local pour les gérer, généralement dans le dossier `.m2/repository`. Ce dossier peut vite prendre beaucoup d'espaces !!
3. Créer votre dossier repository à l'endroit souhaité et modifier le fichier `setting.xml` de votre `.m2`

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0 http://maven.apache.org
/xsd/settings-1.0.0.xsd">
  ...
  <!-- localRepository
  | The path to the local repository maven will use to store artifacts.
  |
  | Default: ${user.home}/.m2/repository -->
  <localRepository>chemin/vers/repertoire/env/maven/repository</localRepository>
  ...
</settings>
```



VARIABLES D'ENVIRONNEMENTS

SOUS WINDOWS

VARIABLES D'ENVIRONNEMENTS

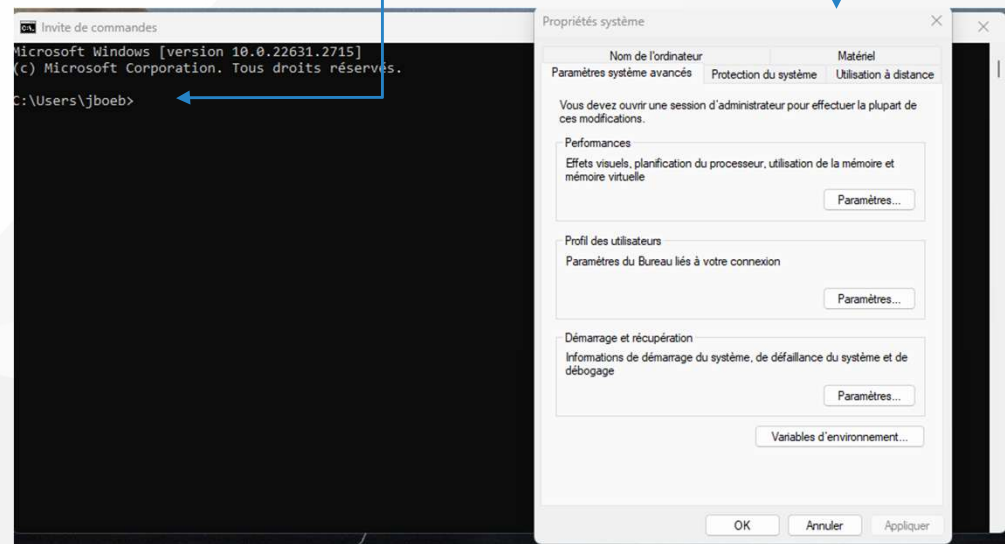
Pour que Maven puisse être utilisé facilement, il nous faut configurer les variables d'environnement de Windows.

Les variables d'environnement sont des variables dynamiques utilisées par les différents processus d'un système d'exploitation (Windows, Unix...). Elles servent à communiquer des informations entre les programmes qui ne se trouvent pas sur la même ligne hiérarchique, et qui ont donc besoin d'une convention pour se communiquer mutuellement leurs choix.

1. Pour pouvoir utiliser Maven depuis un terminal "invite de commandes", il faut connaître l'emplacement de son exécutable.
2. Maven a également besoin de connaître l'emplacement de la JDK pour pouvoir l'utiliser.

Plusieurs solutions sont possibles pour configurer les variables d'environnement sous Windows.

- Depuis un terminal "invite de commandes"
- Depuis les propriétés système de Windows



VARIABLES D'ENVIRONNEMENTS



Avec IntelliJ où se trouve Maven et Java ?

Soit depuis le terminal

Set JAVA_HOME=chemin vers le répertoire d'installation de la JDK

- Set MAVEN_HOME=chemin vers le répertoire d'installation de Maven

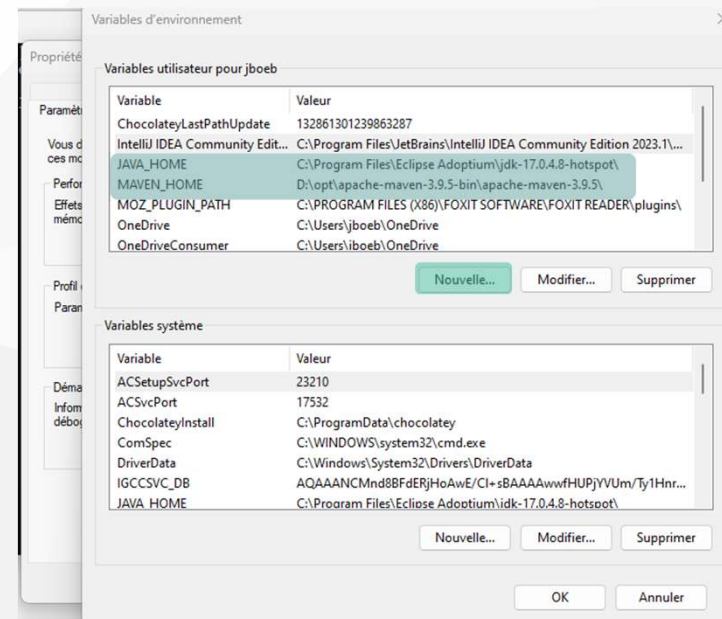
Soit depuis les propriétés système

```

C:\Users\jboeb>set MAVEN_HOME=D:\opt\apache-maven-3.9.5-bin\apache-maven-3.9.5

Microsoft Windows [version 10.0.22631.2715]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\jboeb>set JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-17.0.4.8-hotspot
  
```



VARIABLES D'ENVIRONNEMENTS

Après avoir configuré nos deux nouvelles variables d'environnement, nous pouvons ajouter au PATH, ces deux variables pour indiquer au système où trouver les exécutables.

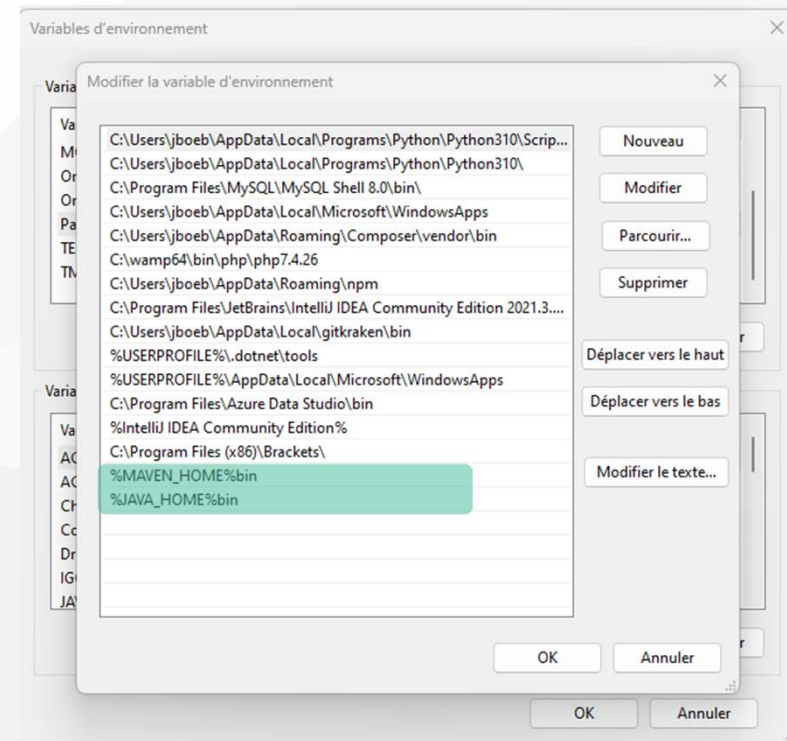
Toujours au choix :

- Soit depuis le terminal
- Soit depuis les propriétés système

ATTENTION, pour le PATH, on rajoute le répertoire bin au chemin. Ce répertoire contient l'emplacement des exécutables de Maven et Java

```

C:\Users\jboeb>PATH=%JAVA_HOME%bin;%MAVEN_HOME%bin
  
```



TESTER SON INSTALLATION

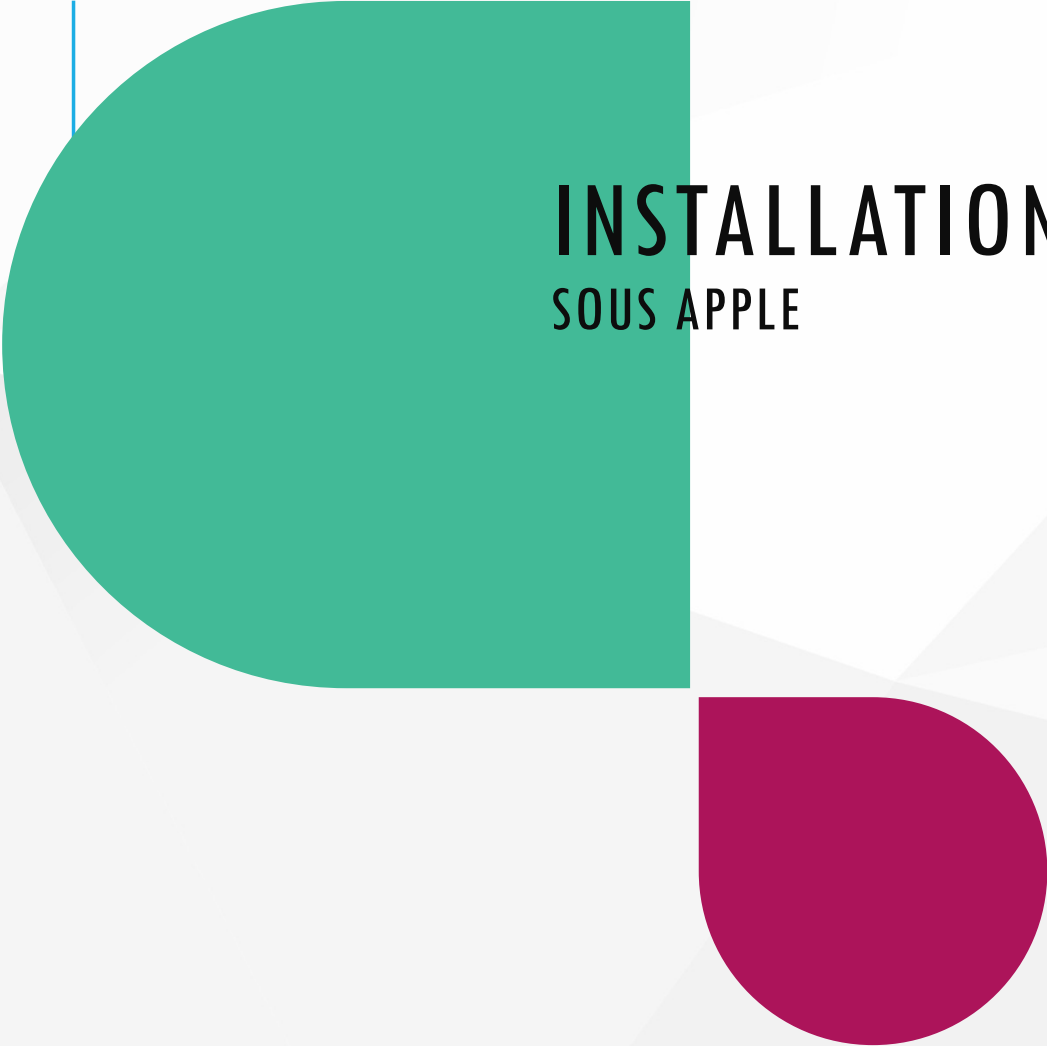
Test installation de Maven

```
jboeb@LAPTOP-RVGOQTRQ MINGW64 ~/Documents/PROJETS/JAVA/MavenDemo/CoursMaven
$ mvn -v
Apache Maven 3.8.7 (b89d5959fcde851dcb1c8946a785a163f14e1e29)
Maven home: C:\opt\apache-maven-3.8.7
Java version: 11.0.15, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-11.0.15.10-hotspot
Default locale: fr_FR, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

Test installation de Java

```
C:\Users\jboeb>java -version
openjdk version "17.0.4" 2022-07-19
OpenJDK Runtime Environment Temurin-17.0.4+8 (build 17.0.4+8)
OpenJDK 64-Bit Server VM Temurin-17.0.4+8 (build 17.0.4+8, mixed mode, sharing)

C:\Users\jboeb>
```



INSTALLATION SOUS MAC

SOUS APPLE

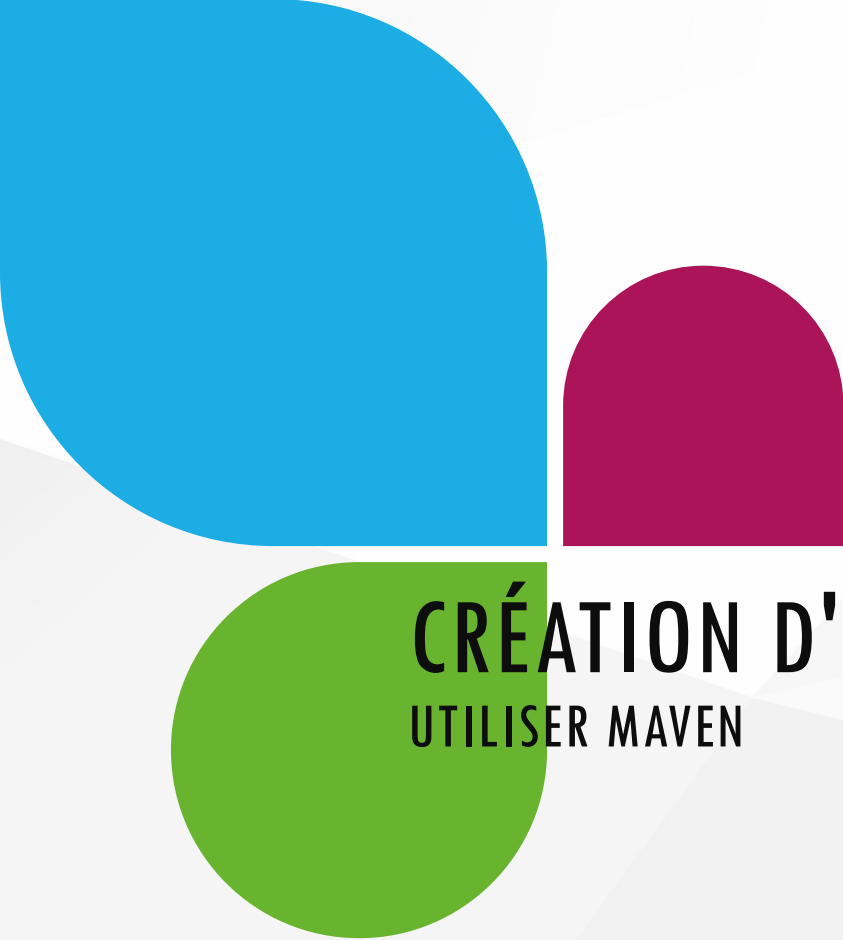
INSTALLATION ET CONFIGURATION

1. Créer une variable d'environnement pour Java
2. Tester l'installation de Java
3. Télécharger l'archi

Installing Maven on Mac OS

- Apache Maven requires JDK to execute.
- Steps to install OpenJDK on Mac OS:
 - Download OpenJDK Binaries ZIP File and Extract it
 - Set up **JAVA_HOME** and **Path** environment variables
 - Verify the installation using **java -version** command.
- Steps to install Maven on Mac OS:
 - Download Maven "Binary tar.gz Archive" and Extract it
 - Set up **M2_HOME**, and **Path** environment variables
 - Verify the installation using **mvn -version** command.





CRÉATION D'UN PROJET

UTILISER MAVEN

CRÉER VOTRE 1^{ER} PROJET MAVEN

Maven utilise une approche dite "**convention over configuration**" qui signifie qu'il a établi un certain nombre de conventions et si respectez, beaucoup de choses seront automatiques.

Une des premières conventions est l'arborescence de Maven.

Celle-ci étant fixé, Maven vous permet de générer votre projet selon des squelette qu'on nomme **archetype**.

- Par exemple, dans le cadre de nos projets desktop, nous utiliserons l'archetype suivant : **maven-archetype-quickstart** ou **maven-archetype-simple**
- On peut lister les archetypes avec **mvn archetype:generate > listeArchetypes.txt**

Pour générer un projet maven, il suffit d'effectuer la commande suivante : **mvn archetype:generate** en lui indiquant l'archetype choisi dans le dossier de votre futur projet.

<https://maven.apache.org/archetypes/index.html>

La commande complète se trouve sur le site de Maven et il suffit simplement de la copier-coller dans le terminal.

Maven Quickstart Archetype

maven-archetype-quickstart is an archetype which generates a sample Maven project:

```
1. project
2. |-- pom.xml
3. |-- src
4. |   |-- main
5. |   |   |-- java
6. |   |   |   |-- $package
7. |   |   |   |   |-- App.java
8. |   |-- test
9. |   |   |-- java
10. |   |   |   |-- $package
11. |   |   |   |   |-- AppTest.java
```

Usage

To generate a new project from this archetype, type:

```
1. mvn archetype:generate -DarchetypeGroupId=org.apache.maven.archetypes -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.4
```

Enfin, on laisse Maven faire son travail...

CRÉER VOTRE 1^{ER} PROJET MAVEN

Maven va vous poser des questions afin de personnaliser le projet :

- **GroupId** : fr.afpa.pompey.cda22045
- **ArtifactId** : Maven Quickstart Archetype
- **Version (0.0.1-SNAPSHOT)** : *laisser par défaut*
- **Package** : *laisser par défaut*
- **Confirmer par Y**

```
[INFO] Generating project in Interactive mode
[INFO] Archetype repository not defined. Using the one from [org.apache.maven.archetypes:maven-archetype-webapp:1.4] found in catalog remote
Define value for property 'groupId': fr.afpa.CDA10
Define value for property 'artifactId': appWeb
Define value for property 'version' 1.0-SNAPSHOT: :
Define value for property 'package' fr.afpa.CDA10: :
Confirm properties configuration:
groupId: fr.afpa.CDA10
artifactId: appWeb
version: 1.0-SNAPSHOT
package: fr.afpa.CDA10
Y: :
```

A l'apparition du message **BUILD SUCCESS**, votre projet est créé.

Avant de voir ce que Maven a généré pour nous, revenons sur quelques points.

- **GroupId** permet d'identifier de manière unique votre projet. Il suit les conventions de nom de package de Java. Cela signifie qu'il commence par un nom de domaine inversé que vous contrôlez. Ici pour l'exemple fr.afpa.pompey.cda22045
- **ArtifactId** est le nom du projet.
- **Version** indique la version en cours de développement. Comme on est en phase de développement par défaut, nous sommes en version **0.0.1-SNAPSHOT** terme qui indique un développement en cours.
- **Package** est le package de votre projet. On laisse donc par défaut la proposition de Maven afin de conserver une cohérence avec le groupId choisi.

TEST

Après la création du projet par Maven, il faut effectuer quelques contrôles.

1. Changer la version de la JDK car par défaut, il se met en version 1.7 dans le pom.xml
2. Compilez votre projet, exécutez les tests et construisez un JAR de votre application en une seule commande : `mvn package`.
3. Si la compilation et tous les tests passent sans encombre, un fichier `target/<artifactId>-<version>.<packaging>` est généré.
4. `java -cp target/AppMaven-1.0-SNAPSHOT.jar fr.afpa.pompey.cda22045.App`

```
<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<!-- mise à jour de la JDK -->
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
</properties>
```

```
jboeb@LAPTOP-RVGOQTRQ MINGW64 /d/PROJETS/JAVA/DemoMaven/AppMaven
$ java -cp target/AppMaven-1.0-SNAPSHOT.jar fr.afpa.pompey.cda22045.App
Hello world! - bonjour
```


NUMÉROTATION DES VERSIONS

<https://semver.org/lang/fr/>

Schéma :

`<major>.<mini>[.<micro>][-<qualifier>[-<buildnumber>]]`

- Incrément
 - **Major** : changement majeur
 - pas de retro-compatibilité (descendante) garantie
 - **Mini** : ajouts fonctionnels
 - retro-compatibilité garantie
 - **Micro** : maintenance corrective (bug fix)
- **Qualificateurs**
 - SNAPSHOT (Maven) : version en évolution
 - alpha1 : version alpha (très instable et incomplète)
 - beta1, b1, b2 : version beta (instable)
 - rc1, rc2 : release candidate
 - m1, m2 : milestone
 - ea : early access
 - 20081014123459001 : date du build
 - jdk5 : dépendance avec une arch, un os, un langage
- **Ordre sur les versions**
 - Différent de l'ordre lexicographique
 - 1.1.1 < 1.1.2 < 1.2.2
 - 1.1.1-SNAPSHOT < 1.1.1
 - 1.1.1-alpha1 < 1.1.1-alpha2 < 1.1.1-b1 < 1.1.1-rc1 < 1.1.1-rc2 < 1.1.1
- **Remarque (parfois)**
 - <mini> pair : release stable
 - <mini> impair : release instable



DÉCOUVERTE DU POM

POM.XML

DECRIRE SON PROJET

Tout se passe – ou presque – dans le fichier pom.xml. Je suis sûr que vous l'aviez deviné !

Ce fichier, au format XML, permet de définir le POM (Project Object Model) utilisé par Maven.

- <https://maven.apache.org/pom.html>

- les coordonnées du projet Maven
 - Un premier bloc définit les informations du projet relatives à Maven, avec :
 - **groupId** : identifiant de l'organisation gérant le projet. Cet identifiant reprend la notation des packages Java. En général, celui-ci correspond au package de base de l'application, mais ce n'est pas obligatoire.
 - **artifactId** : identifiant du projet
 - **version** : version du projet.
 - **packaging** : type de packaging devant être généré par Maven (jar, war, ear...).
- les informations générales du projet Maven
 - Les informations générales du projet, avec les balises :
 - **name** : le nom du projet
 - **description** : la description du projet
 - **url** : URL du projet ou de l'application en production.

DECRIRE SON PROJET

Tout se passe – ou presque – dans le fichier pom.xml. Je suis sûr que vous l'aviez deviné ! Ce fichier, au format XML, permet de définir le POM (Project Object Model) utilisé par Maven.

- <https://maven.apache.org/pom.html>

- l'organisation du projet

```
<!-- ===== Organisation ===== -->
<organization>
  <name>Mon Entreprise</name>
  <url>http://www.exemple.org</url>
</organization>
```

- les licences du projet

```
<licenses>
  <license>
    <name>Apache License, Version 2.0</name>
    <url>https://www.apache.org/licenses/LICENSE-2.0.txt</url>
  </license>
</licenses>
```



DÉPENDANCES

AJOUTER UNE LIBRAIRIE

GESTION DES DÉPENDANCES

Une des forces de Maven est sa puissance dans la gestion des dépendances.

En effet, grâce à Maven, vous pouvez facilement ajouter des bibliothèques tierces à votre application.

1. Ajouter une dépendance

- Pour ajouter une dépendance, il suffit d'ajouter une balise `<dependency>` à votre section `<dependencies>` dans votre pom.xml

2. Où trouver une dépendance ?

- Maven utilise des repositories. Un repository est un endroit où sont stockées des dépendances sous un format imposé par Maven.
- Par défaut, Maven utilise le repository central à l'adresse : <https://mvnrepository.com/>
- Depuis le repository, il suffit simplement depuis le module de recherche, indiquer la dépendance recherchée et de recopier le code fourni dans votre pom.xml

3. Transitivité des dépendances

- Maven gère également la transitivité des dépendances. Cela signifie que Maven est capable d'ajouter automatiquement les dépendances requises par les dépendances que vous avez ajoutées.

GESTION DES DÉPENDANCES

Options dans la gestion de dépendances

- Ignorer des sous-dépendances
 - Si on a des dépendances telles que $A \rightarrow B \rightarrow C \rightarrow D$, alors A dépendra de B, C et D. Si A n'utilise qu'une petite partie de B qui n'a pas besoin de C, alors il est possible d'indiquer à Maven d'ignorer la dépendance $B \rightarrow C$ dans A. A ne dépendra alors que de B et plus de C et D.
 - Pour cela, la balise `<exclusion>` permet d'indiquer la dépendance qu'on ne souhaite pas prendre.
- Définir la portée d'une dépendance
 - Il se peut que la dépendance que vous voulez ajouter ne soit liée qu'à un contexte particulier comme l'exécution des tests par exemple.
 - Pour indiquer cela à Maven, les dépendances sont attachées à un **scope**.
 - Par défaut ce scope est **compile**. Ce scope indique que la dépendance est utilisée lors de la compilation et sera accessible dans tous les contextes.

Scope	Transitif*	Compilation	Test	Exécution
compile	✓	✓	✓	✓
test			✓	
provided		✓	✓	
runtime	✓		✓	✓

➤ X dépend de Y

- transitif (*compile* ou *runtime*), Maven ajoute la dépendance vers Y à votre projet
- **non transitif** (*test* ou *provided*), Maven **n'ajoute pas** la dépendance vers Y à votre projet

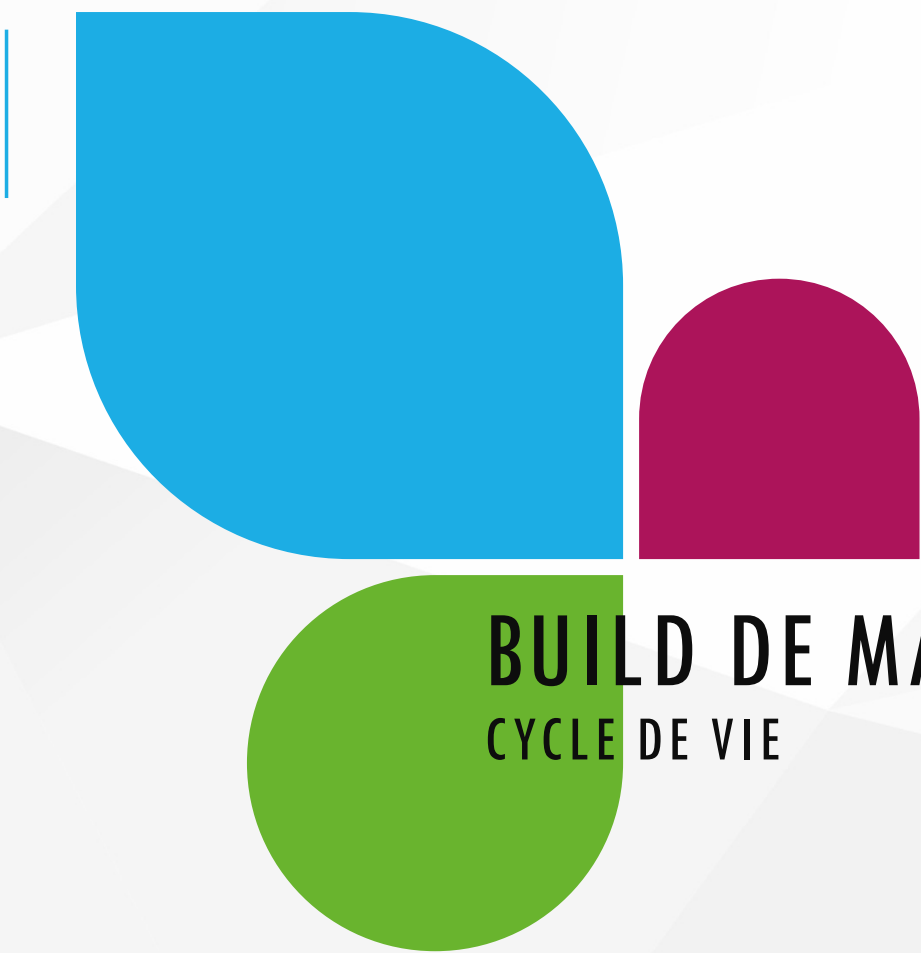
GESTION DES DÉPENDANCES

Gérer les dépendances de manière globale.

- Avec la balise `<dependencyManagement>`, vous pouvez gérer de manière globale, dans le cadre de projet contenant des modules, les dépendances.
- Cela peut être le cas notamment dans l'utilisation de framework (Spring, Apache Struts ...).
 - Dans ce cas au lieu d'importer dépendances par dépendances, on peut utiliser un [Bill of Materials \(BOM\)](#).
 - Il s'agit d'un fichier POM spécifiques contenant les dépendances du framework

```
<dependencyManagement>
  <dependencies>
    ...
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.12</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>javax.validation</groupId>
      <artifactId>validation-api</artifactId>
      <version>1.1.0.Final</version>
    </dependency>
    <dependency>
      <groupId>org.apache.bval</groupId>
      <artifactId>bval-jsr</artifactId>
      <version>1.1.2</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>servlet-api</artifactId>
      <version>2.5</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-framework-bom</artifactId>
      <version>4.3.11.RELEASE</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
  ...
</dependencyManagement>
```

BUILD DE MAVEN

CYCLE DE VIE

CYCLE DE VIE DE MAVEN

Le cycle de vie de Maven est une séquence d'étapes à suivre pour construire un projet.

Afin d'automatiser la construction d'un projet, Maven s'appuie sur ces cycles de vie de construction appelés **build lifecycle** dans le jargon de Maven.

Maven fonctionne avec 3 cycles de vie.

- **CLEAN** : permet de nettoyer le projet
 - **SITE** : permet de créer un site web du projet
 - **DEFAULT** : permet de construire et déployer le projet.
- Ces **build lifecycles** sont découpés en phases qui sont exécutées séquentiellement les unes à la suite des autres.

1. CLEAN

1. pre-clean
2. clean
3. post-clean



2. DEFAULT (principaux)

1. Validate
2. compile
3. test-compile
4. Test
5. Package
6. Integration-test
7. verify
8. Install
9. Deploy



3. SITE

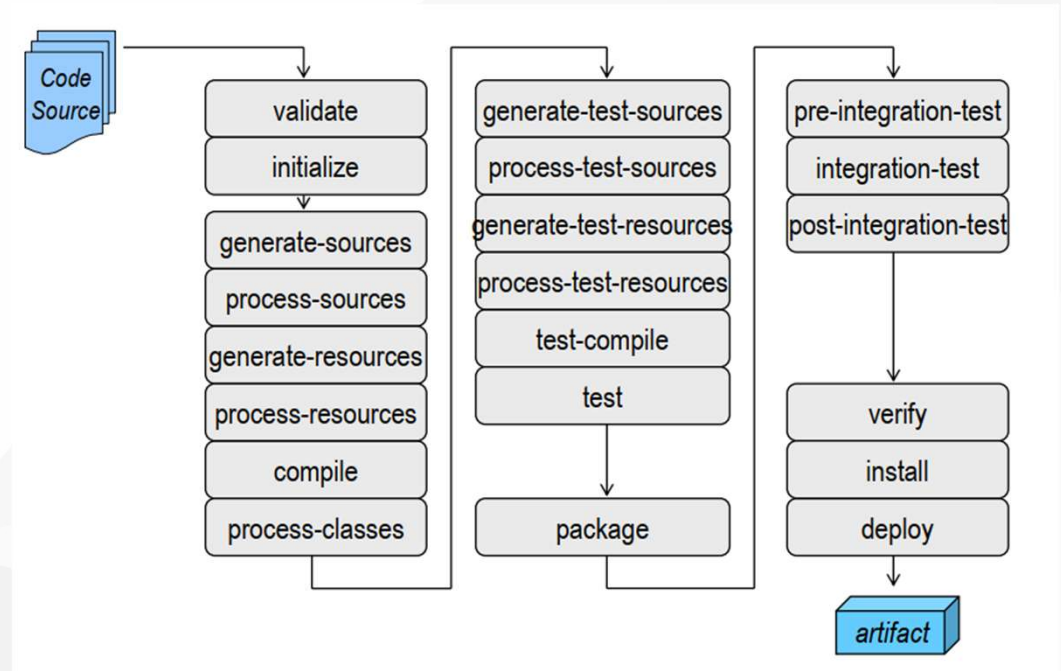
1. pre-site
2. site
3. post-site
4. site-deploy



<https://maven.apache.org/ref/3.9.0/maven-core/lifecycles.html>

CYCLE DE VIE DE MAVEN

Séquence de 21 phases



LANCEMENT UN BUILD LIFECYCLE

Quand vous lancez une construction Maven en ligne de commande, vous précisez simplement une phase d'un des build lifecycles et Maven se charge d'exécuter, dans l'ordre, toutes les phases qui composent le build lifecycle jusqu'à la phase indiquée.

- `mvn package` ?
 - Eh bien dans cette commande, vous indiquez la phase package du build lifecycle default. Maven va alors exécuter les phases `validate`, `compile`, `test` et enfin package.
 - Les pre-xxx ne seront pas montrés lors de l'affichage. Elles sont lancées implicitement.
- `mvn clean package` ?
 - Eh bien dans cette commande, vous indiquez la phase clean du build lifecycle clean puis package du build lifecycle default. Maven va alors exécuter les phases clean `validate`, `compile`, `test` et enfin `package`.

BUILD LYFECYCLE DEFAULT

Le build lifecycle **Default** est celui que nous allons le plus utilisé avec maven car il est composé des phases de construction du projet

Nom	Utilité
Validate	Valider le pom.xml
Compile	Compiler les fichiers .java en .class dans le dossier /target
Test	Lancer les tests unitaires dans /src/test/java
Package	Créer un package (un fichier « .jar », « .war », « .ear »)
Verify	Lancer des tests pour vérifier la validité du package
Install	Déployer le package vers le repository local afin qu'il puisse être accessible dans d'autres projets
Deploy	Déployer le package vers un repository distant

LES GOALS

Nous avons vu qu'un **build lifecycle** est constitué d'une série de phases.

- Mais la granularité de l'exécution de Maven est encore plus fine.

En effet, les phases sont découpées en tâches.

- Chaque tâche est assurée par un plugin Maven.

Dans le jargon de Maven, ces tâches s'appellent des **goals**.

- Par exemple, la phase **test** est réalisée par défaut par le goal **surefire:test**, c'est-à-dire le goal test du plugin **surefire**.
- Maven fournit de base un certain nombre de plugins.

```
<plugin>
  <artifactId>maven-clean-plugin</artifactId>
  <version>3.1.0</version>
</plugin>
<plugin>
  <artifactId>maven-site-plugin</artifactId>
  <version>3.7.1</version>
</plugin>
<plugin>
  <artifactId>maven-project-info-reports-plugin</artifactId>
  <version>3.0.0</version>
</plugin>
<!-- see http://maven.apache.org/ref/current/maven-core/default<
<plugin>
  <artifactId>maven-resources-plugin</artifactId>
  <version>3.0.2</version>
</plugin>
<plugin>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.0</version>
</plugin>
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>2.22.1</version>
</plugin>
<plugin>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.0.2</version>
</plugin>
<plugin>
  <artifactId>maven-install-plugin</artifactId>
  <version>2.5.2</version>
</plugin>
<plugin>
  <artifactId>maven-deploy-plugin</artifactId>
  <version>2.8.2</version>
</plugin>
```

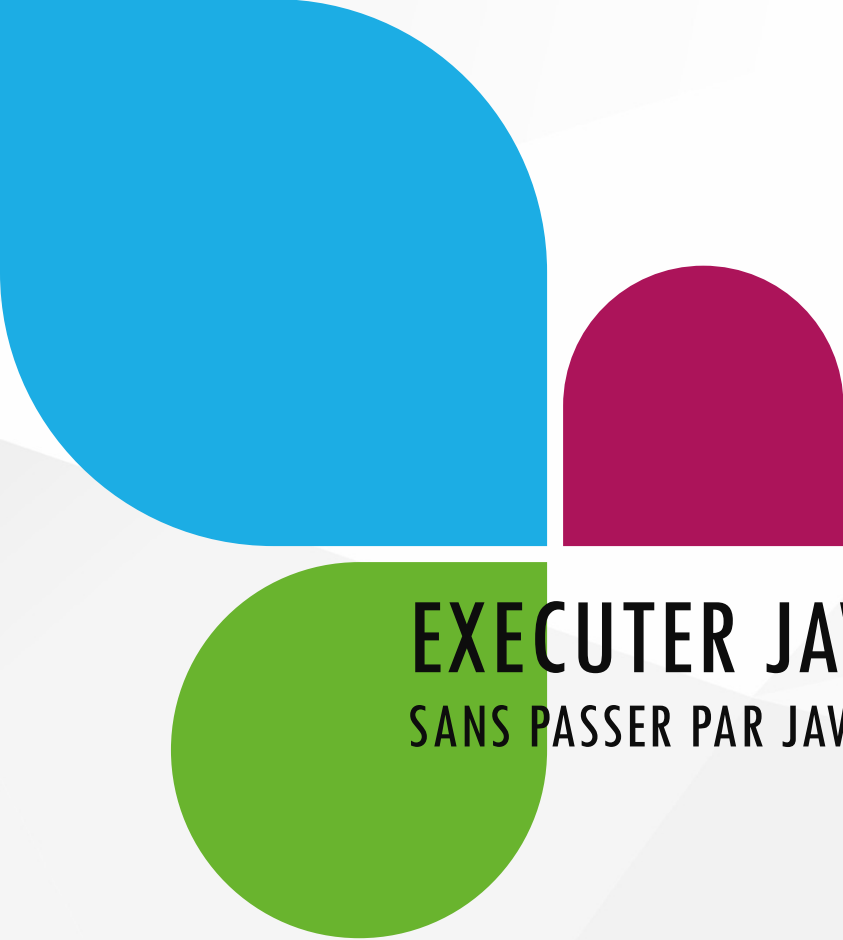
AFFINER SON BUILD

On peut par diverses options affiner les **goals**.

Par exemple, je souhaite lors de la compilation afficher les méthodes dépréciées et les avertissements.

- En parcourant la documentation du goal compile, on obtient les différentes options :
<https://maven.apache.org/plugins/maven-compiler-plugin/compile-mojo.html>

```
<plugin>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.0</version>
  <configuration>
    <showDeprecation>true</showDeprecation>
    <showWarnings>true</showWarnings>
  </configuration>
</plugin>
```



EXECUTER JAVA DEPUIS MAVEN

SANS PASSER PAR JAVA

EXECUTER DU JAVA AVEC MAVEN

Pour cela, dans la partie Build de votre pom.xml, il suffit de rajouter simplement un nouveau plugin exec-maven-plugin et lui indiquer votre classe principale.

Attention, ce plugin se place après le bloc `<pluginManagement>...</pluginManagement>` du build

Ensuite, depuis le terminal, vous lancez la commande : `mvn exec:java`

```
</pluginManagement>

<plugins>
  <!-- plugin pour que maven execute java -->
  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.1.0</version>
    <configuration>
      <mainClass>training.afpa.cda24060.App</mainClass>
    </configuration>
  </plugin>
</plugins>
</build>
```

```
PS D:\PROJETS\CDA24060\JAVA\Maven\demomaven> mvn exec:java
Hello World!
JSON généré : {"name":"John Doe","age":30}
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.624 s
[INFO] Finished at: 2025-11-04T11:17:03+01:00
[INFO] -----
```



CONSTRUIRE UN PACKAGE COMPLET

AVEC LES DEPENDANCES.

CONSTRUIRE UN PACKAGE COMPLET

Maven télécharge les dépendances dans le repository local (~/.m2/repository), mais le mvn package par défaut ne les met pas dans le JAR.

Plusieurs solutions sont possibles :

- Option 1 — Utiliser le *maven-assembly-plugin* (faire un « fat JAR »)
- Option 2 — Utiliser le *maven-shade-plugin* (encore mieux pour la production)

Attention, ce plugin se place après le bloc `<pluginManagement>...</pluginManagement>` du build

- Ensuite `mvn clean package` et cette fois-ci, on utilise `java -jar nom_du_jar`

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.5.1</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>shade</goal>
      </goals>
      <configuration>
        <transformers>
          <transformer implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>training.afpa.cda24060.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution>
  </executions>
</plugin>
```

```
PS D:\PROJETS\CDA24060\JAVA\Maven\demomaven> java -jar .\target\demomaven-1.0-SNAPSHOT.jar
Hello World!
JSON généré : {"name":"John Doe","age":30}
```

IMPORTANT

Il est important de comprendre qu'avec Maven, vous pouvez affiner votre construction de projets selon vos besoins et c'est la force de Maven

Par la suite, vous allez sans doute passer par l'IDE, ce qui est normal, mais gardez à l'esprit qu'au travers de l'IDE, il va effectuer beaucoup de traitement à votre place notamment au niveau du build... mais qui ne vous empêche pas de paramétrer Maven selon vos besoins.





GÉNÉRER UN SITE DE VOTRE PROJET

DOCUMENTER VOTRE PROJET

GÉNÉRER UN SITE DE VOTRE PROJET

Dans ce chapitre, je vais vous montrer qu'il est aussi possible d'utiliser Maven pour générer un site web pour votre projet.

Vous y retrouverez :

- des informations générales sur celui-ci,
- la Javadoc,
- des rapports sur l'exécution des tests, la qualité du code...

- Configuration de base du site
- Par convention, les sources servant à la génération du site du projet se trouvent dans le répertoire `src/site` du dossier target.

```
1  src
2  └─ site
3      └─ apt
4          └─ xxx.apt
5              ...
6      └─ fml
7          └─ faq.fml
8              ...
9      └─ markdown
10         └─ yyy.md
11             ...
12     └─ ...
13     └─ resources
14         └─ css
15             └─ style.css
16         └─ img
17             └─ zzz.png
18                 ...
19     └─ ...
20     └─ site.xml
```

- Le fichier `src/site/site.xml` est le site descriptor.
 - Il permet de définir la structure du site.
- Les répertoires `apt`, `markdown`, `fml`... hébergent les fichiers source des pages du site qui seront converties en HTML par Maven.
- Ces répertoires accueillent des fichiers de leur format respectif :
 - `apt` : Format Almost Plain Text
 - `fml` : Format FAQ Markup Language
 - `markdown` : Format Markdown
 - autres formats supportés par Maven
- Le répertoire `resources` pour les autres ressources :
 - `css/style.css` : fichier CSS permettant d'ajuster le style par défaut
 - ...

CYCLE DE VIE : SITE

Il est un peu à part et est fait des étapes suivantes :

- `presite` : exécute les différents processus nécessaires à la génération du site
- `site` : génère le site du projet
- `postsite` : prépare le site pour le déploiement
- `site-deploy` : déploie le site généré vers un serveur web.

Toutes ces étapes sont réalisées au moyen du plugin suivant : `maven-site-plugin`

Ce plugin possède les goals suivants :

- `site:site` : permet de générer le site pour un projet. Les liens entre les modules ne vont pas fonctionner pour un projet multimodule
- `site:deploy` : déploie le site sur un serveur web
- `site:run` : lance le site en utilisant un serveur Jetty
- `site:stage` : génère le site localement comme s'il le déployait sur un serveur. Ce goal permet de tester si les liens entre modules fonctionnent correctement
- `site:stage-deploy` : déploie le site local vers un serveur web
- `site:attach-descriptor` : ajout le fichier de description du site (`site.xml`) dans la liste des éléments à déployer/installer
- `site:jar` : compresse la totalité du site dans un fichier `.jar`.

INFORMATIONS SUR LE PROJET

La création de la documentation via Maven va permettre d'afficher tout un ensemble d'informations, dont des rapports et même de la déployer si besoin.

- Membres du projet
- Intégration continue
- Gestion des demandes
- Licence
- Description et résumé du projet
- Contrôle de sources
- Listes de diffusion
- Pages gérées par Maven
- des rapports :
 - PMD : Analyse des classes du projet
 - Checkstyle : Contrôle qualité
 - Tests unitaires : Analyse des tests
 - Couverture de codes
 - Javadoc : génération de la Javadoc

POM.XML

C'est avec le fichier pom.xml et les différents blocs descriptifs que nous allons pouvoir alimenter le site généré.

En effectuant un simple `mvn site`, vous effectuez une génération du site par maven.

- Par défaut, le site est en anglais mais on peut indiquer à maven de le générer en français.

```
<plugin>
  <artifactId>maven-site-plugin</artifactId>
  <version>3.12.1</version>
  <configuration>
    <!-- en français -->
    <locales>fr</locales>
  </configuration>
</plugin>
```

UNE DES INFORMATIONS QUE NOUS POUVONS FAIRE APPARAÎTRE SONT LES MEMBRES ET CONTRIBUTEURS DU PROJET.

L'équipe

Un projet réussi exige que beaucoup de personnes jouent plusieurs rôles.

L'équipe est composée des membres et des contributeurs. Les membres sont des personnes qui ont contribué au projet. Le nombre de contributeurs au projet est illimité. Tout

Membres

Ce qui suit est la liste des développeurs avec leurs rôles ayant contribué au projet.

Image	Id	Nom	Rôle
	neojero	Jérôme BOEBION	Formateur

Contributeurs

Les personnes additionnelles suivantes ont contribué à ce projet.

Image	Nom	Courriel
	John Doe	john@john-doe.com

```
<developers>
  <developer>
    <id>neojero</id>
    <name>Jérôme BOEBION</name>
    <email>jerome.boebion@afpa.fr</email>
    <url>https://www.boebion.fr</url>
    <organization>https://afpa-dev-pompey.fr</organization>
    <organizationUrl>https://www.afpa.fr</organizationUrl>
    <roles>
      <role>Formateur</role>
      <role>Développeur</role>
    </roles>
    <timezone>1</timezone>
  </developer>
</developers>
```

```
<contributors>
  <contributor>
    <name>John Doe</name>
    <email>john@john-doe.com</email>
    <url>https://john-doe.com</url>
    <organization>Doe Corporation</organization>
    <organizationUrl>https://john-doe.com</organizationUrl>
    <roles>
      <role>Testeur</role>
    </roles>
    <timezone>1</timezone>
  </contributor>
</contributors>
```

AFFICHER LE GESTIONNAIRE DE SOURCES

On peut aussi afficher un lien vers votre dépôt GitHub par exemple

Vue d'ensemble

This project uses Git to manage its source code. Instructions on Git use can be found at <https://git-scm.com/documentation>.

Accès Web

Ce qui suit est le lien online du dépôt de sources.

<https://github.com/neojero/intellijWeb.git>

Accès de façon anonyme

The source can be checked out anonymously from Git with this command (See <https://git-scm.com/docs/git-clone>):

```
$ git clone --branch Main https://github.com/neojero/intellijWeb.git
```

Accès derrière un firewall

Référez-vous à la documentation du dépôt d'archive utilisé pour plus d'informations sur l'accès derrière un firewall.

```
<scm>
  <connection>scm:git:https://github.com/neojero/intellijWeb.git</connection>
  <url>https://github.com/neojero/intellijWeb.git</url>
  <tag>Main</tag>
</scm>
```

Vue d'ensemble

Ce projet utilise GitHub Issues pour gérer ses issues.

Contrôle des livraisons

Des issues (bogues, dispositifs, demandes de changement

<https://github.com/neojero/intellijWeb/issues>

Vue d'ensemble

Ce projet utilise Jenkins.

Accès

Le lien suivant est celui du système d'intégration continue pour ce projet.

<http://127.0.0.1:9090/continuum>

Notification

Configuration pour notifier les développeurs ou les utilisateurs qu'un build a échoué.

Type	Adresse
mail	devops@afpa-dev-pompey.fr

ON PEUT ÉGALEMENT FAIRE APPARAÎTRE LE
SIGNALEMENT DE BUG OU L'INTÉGRATION
CONTINUE

```
<ciManagement>
  <system>Jenkins</system>
  <url>http://127.0.0.1:9090/jenkins</url>
  <notifiers>
    <notifier>
      <type>mail</type>
      <address>devops@afpa-dev-pompey.fr</address>
      <sendOnError>true</sendOnError>
      <sendOnFailure>true</sendOnFailure>
      <sendOnSuccess>false</sendOnSuccess>
      <sendOnWarning>false</sendOnWarning>
      <configuration>
        <address>devops@afpa-dev-pompey.fr</address>
      </configuration>
    </notifier>
  </notifiers>
</ciManagement>

<issueManagement>
  <system>GitHub Issues</system>
  <url>https://github.com/neojero/intellijWeb/issues</url>
</issueManagement>
```

AFFICHER LES LICENCES DU PROJET

```
<licenses>
  <license>
    <name>GNU GPL V3</name>
    <url>http://www.gnu.org/licenses/gpl-3.0.html</url>
    <distribution>repo</distribution>
    <comments>Une license peu restrictive</comments>
  </license>
  <license>
    <name>Apache Software License, Version 2.0</name>
    <url>http://www.apache.org/licenses/LICENSE-2.0</url>
    <distribution>repo</distribution>
    <comments>La licence de Maven</comments>
  </license>
</licenses>
```

Vue d'ensemble

Typiquement les licences énumérés pour ce projet sont celles du projet lui-même.

Licence du projet

Ce projet est publié sous plusieurs licences :

- [GNU GPL V3](#)
- [Apache Software License, Version 2.0](#)

GNU GPL V3

Une license peu restrictive

[\[Original text\]](#) 

Copy of the license follows:

[Skip to main text](#)

Free Software Supporter:

[JOIN THE FSF](#)



GNU Operating System

Supported by the Free Software Foundation



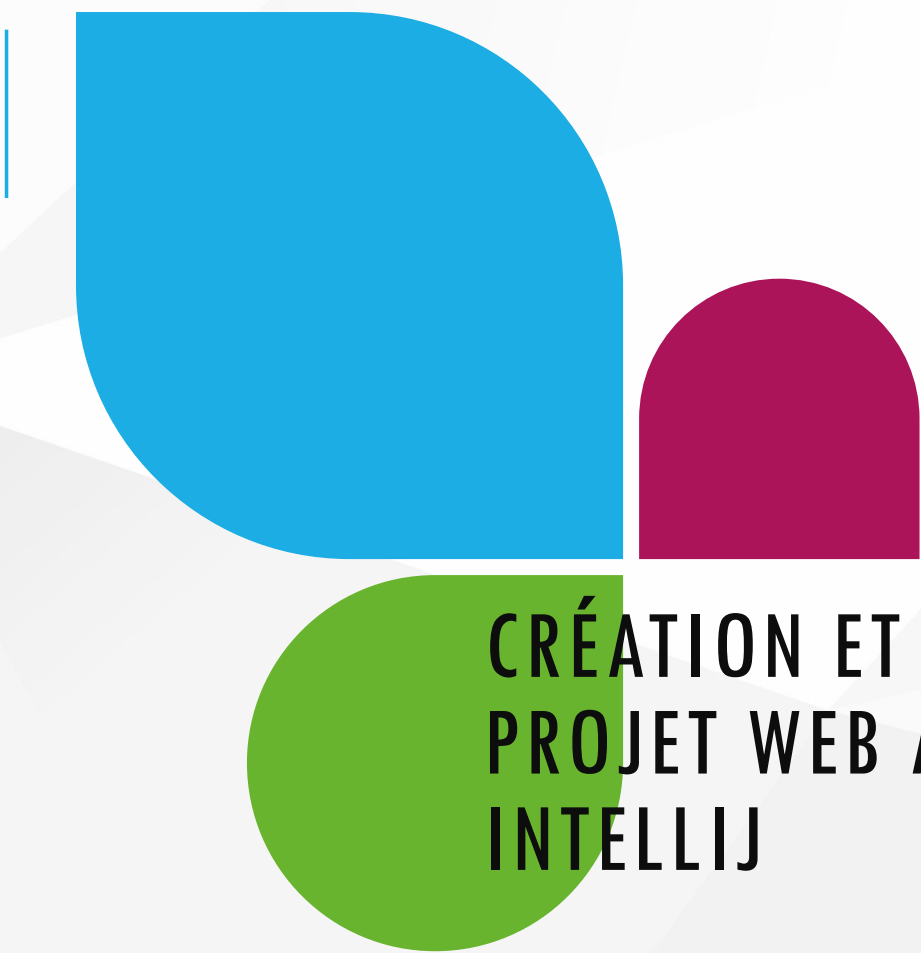
Site navigation [Skip](#)

LA GÉNÉRATION DU SITE.

Afin que maven puisse créer votre site, il faut dans la rubrique `<reporting>...</reporting>` de votre pom.xml, lui fournir les éléments à générer.

Voici un exemple où on affiche .

```
<reporting>
  <plugins>
    <!-- plugin de génération des rapports -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-report-plugin</artifactId>
      <version>3.3.0</version>
    </plugin>
    <!-- Pour les rapports de tests -->
    <plugin>
      <artifactId>maven-project-info-reports-plugin</artifactId>
      <version>3.3.0</version>
      <configuration>
        <showSuccess>true</showSuccess> <!-- Affiche les tests réussis -->
        <showFailureDetail>true</showFailureDetail> <!-- Détails des échecs -->
      </configuration>
    </plugin>
  </plugins>
</reporting>
```



CRÉATION ET CONFIGURATION D'UN PROJET WEB AVEC L'IDE INTELLIJ

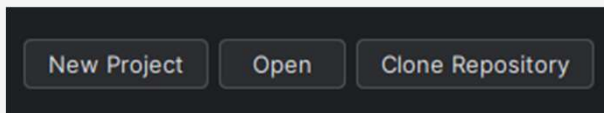
INTELLIJ ET MAVEN

CRÉATION

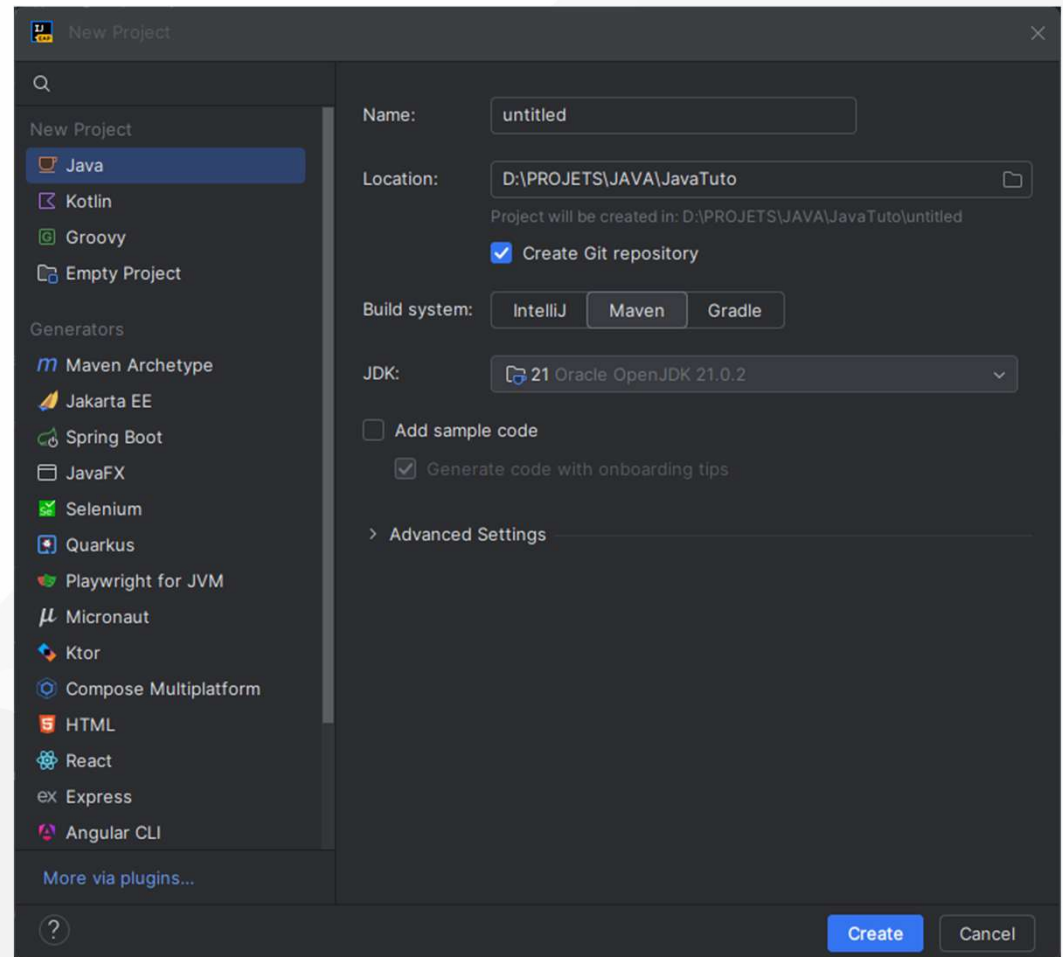
Maven est intégré à IntelliJ et par conséquent, vous épargne l'installation.

Cependant, il est bon de connaître son environnement de travail, de savoir le configurer et de maîtriser les outils qu'on manipule...

Au lancement de votre IDE, vous allez donc créer un nouveau projet...



qui va vous ouvrir la fenêtre de construction du projet :

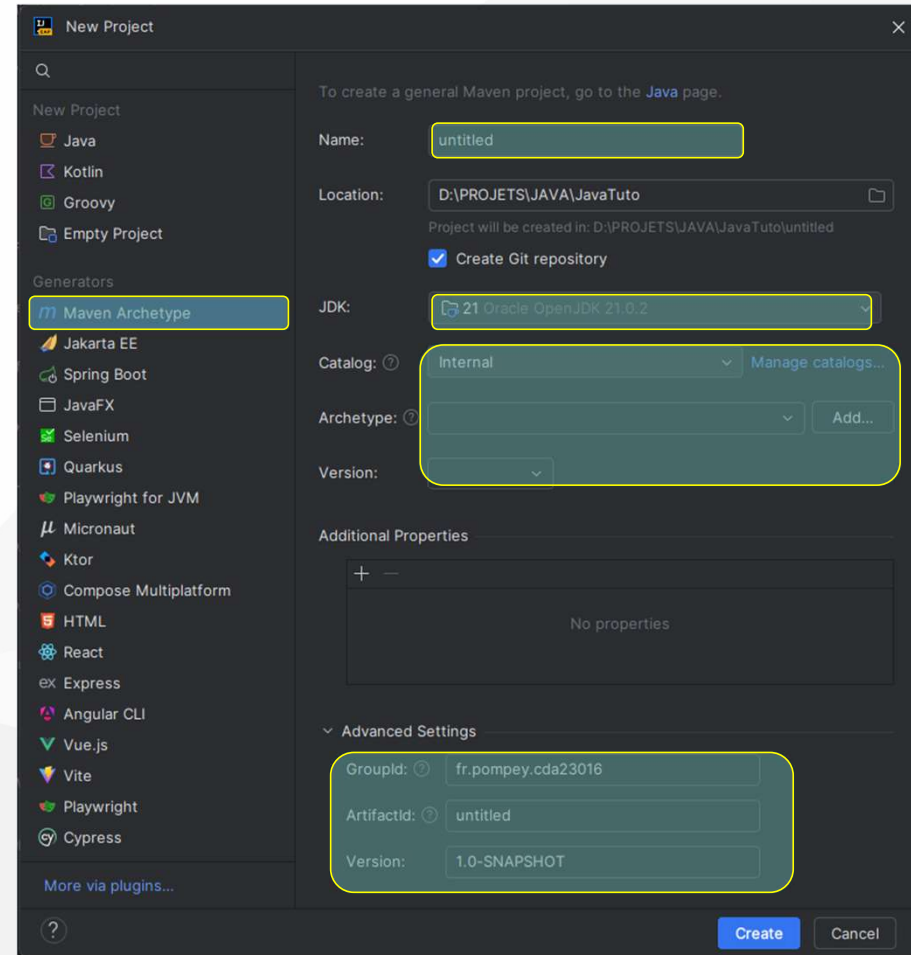


INTELLIJ ET MAVEN

CRÉATION

2 choix possibles :

- Soit construire son projet en partant d'une base maven
- Soit construire son projet en partant d'un archetype maven existant.
 - Name : nom du projet
 - JDK : choix de la JDK
 - Archetype : choix de l'archetype Maven
 - Catalog : choix du catalogue des archetypes
 - Advanced Settings :
 - GroupId
 - ArtifactId
 - Version

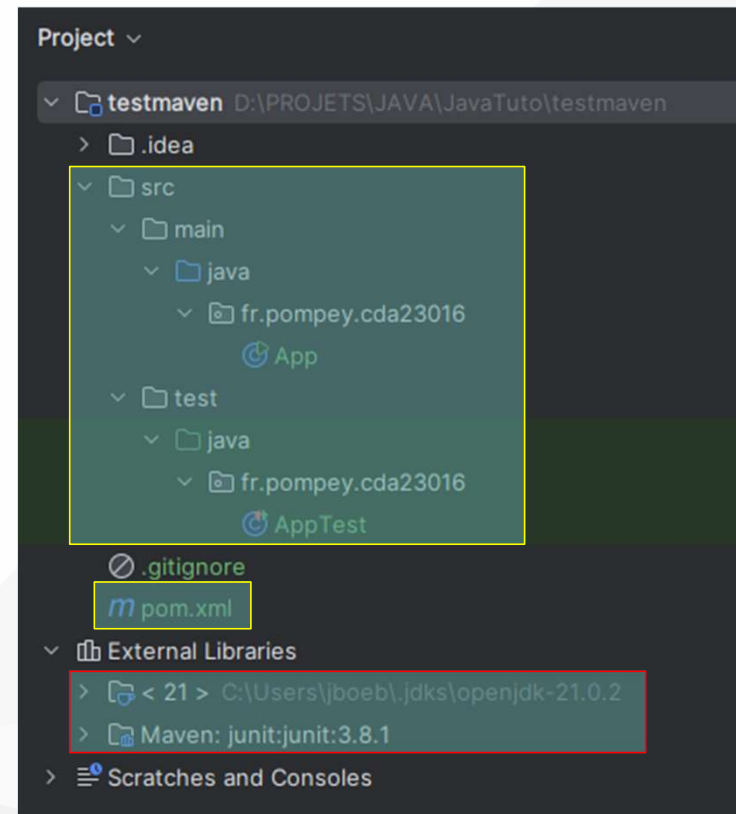


INTELLIJ ET MAVEN

STRUCTURE DU PROJET

Après avoir cliqué sur [create](#) et laissé Maven mettre en place la structure du projet, vous obtenez le message ci-dessous et la structure du projet ci-contre :

```
[INFO] Parameter: basedir, Value: C:\Users\jboe
[INFO] Parameter: package, Value: fr.pompey.cda
[INFO] Parameter: groupId, Value: fr.pompey.cda
[INFO] Parameter: artifactId, Value: testmaven
[INFO] Parameter: packageName, Value: fr.pompey
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.034 s
[INFO] Finished at: 2024-10-30T10:30:34+01:00
[INFO] -----
```



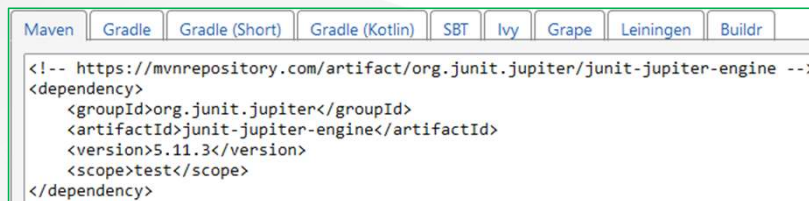
On constate une structure de base conforme aux conventions de Java. Un seul souci.. La version de Junit que nous allons corriger de suite...

INTELLIJ ET MAVEN

LE POM DU PROJET

Afin de corriger notre problème de version de Junit, il suffit dès lors de consulter le Pom.xml du projet.

Pour cela, rien de plus simple, en consultant le repository de Maven, on a juste à récupérer la version de Junit qui nous intéresse :



```

Maven | Gradle | Gradle (Short) | Gradle (Kotlin) | SBT | Ivy | Grape | Leiningen | Buildr
<!-- https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-engine -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.11.3</version>
  <scope>test</scope>
</dependency>
  
```

De remplacer la dépendance dans le Pom.xml et de recharger le projet et de corriger votre classe de tests.

```

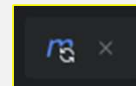
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>fr.pompey.cda23016</groupId>
  <artifactId>testmaven</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>testmaven</name>
  <url>fr.pompey.cda23016/testmaven.fr</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
  
```



Ce petit symbole, qui apparaît en haut à droite, vous indique une modification du POM et donc la nécessité de le recharger.

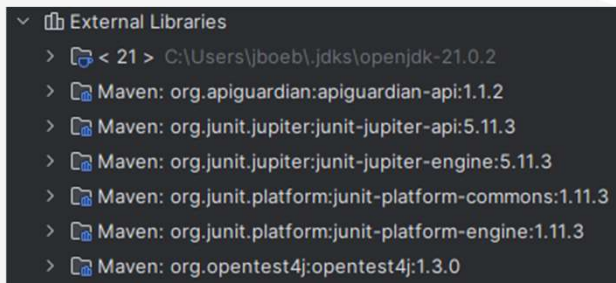
INTELLIJ ET MAVEN

LE POM DU PROJET

Si vous voulez écrire des tests avec Jupiter, ajoutez l'artefact `junit-jupiter-engine` aux dépendances dans le POM.

Cela va ajouter toutes les dépendances requises.

Parmi ces dépendances, `junit-jupiter-api` contient les classes et interfaces que votre source de test doit compiler. `junit-platform-engine` est également ajouté.



```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apac
<modelVersion>4.0.0</modelVersion>

<groupId>fr.pompey.cda23016</groupId>
<artifactId>testmaven</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>testmaven</name>
<url>fr.pompey.cda23016/testmaven.fr</url>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.11.3</version>
    <scope>test</scope>
  </dependency>

</dependencies>
</project>
```

INTELLIJ ET MAVEN

TEST ET BUILD DU PROJET

Une dernière étape est de tester notre projet afin de voir si tout se passe bien.

Pour cela, on va créer un package du projet avec Maven.

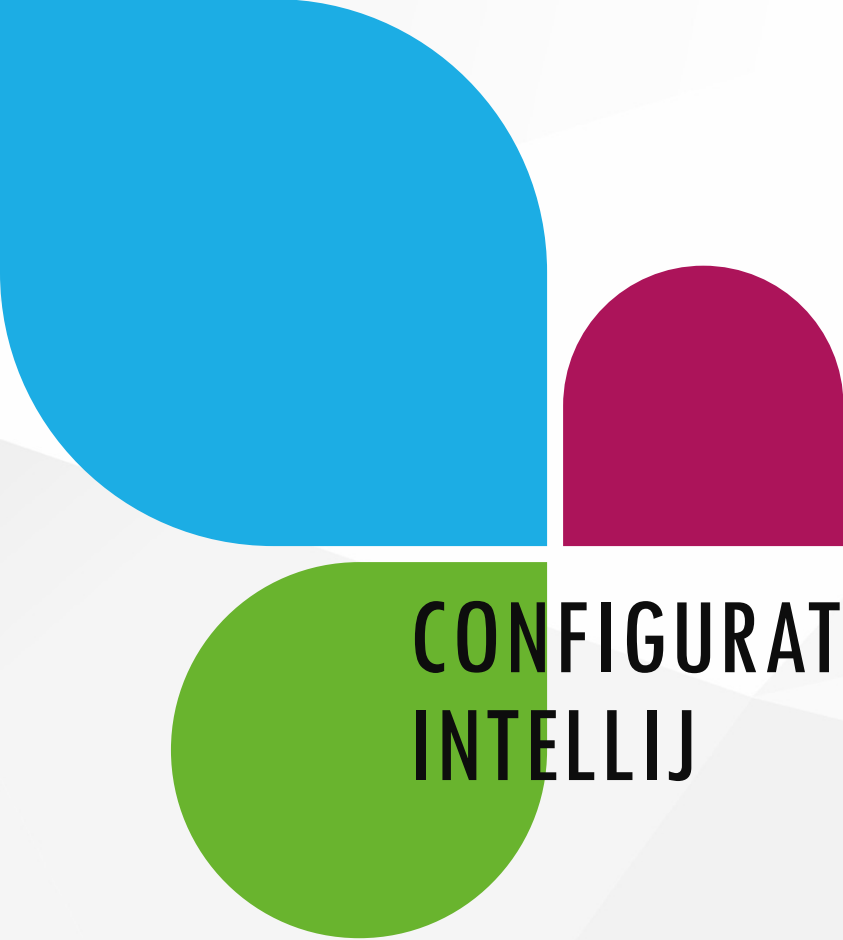
Au sein de l'IDE, vous avez accès aux différentes actions possible avec Maven.

En effectuant un **package**, maven construit votre projet : **clean** → **validate** → **compile** → **test** → **package**

- Si l'un des phases ne se passe pas bien alors le build est arrêté sur la phase en erreur.
- La force de Maven, c'est qu'il permet de faire à chaque build, la phase de tests sans effort !!!

The screenshot shows the IntelliJ IDEA interface. The Maven tool window on the right displays the 'Lifecycle' section with various phases: clean, validate, compile, test, package, verify, install, site, and deploy. The 'package' phase is highlighted. A blue arrow points from the 'package' phase in the Maven window to the terminal output below.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running fr.pompey.cda23016.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in fr.pompey.cda23016.AppTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.4.1:jar (default-jar) @ testmaven ---
[INFO] Building jar: D:\PROJETS\JAVA\JavaTuto\testmaven\target\testmaven-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.336 s
[INFO] Finished at: 2024-10-30T11:50:45+01:00
[INFO] -----
```

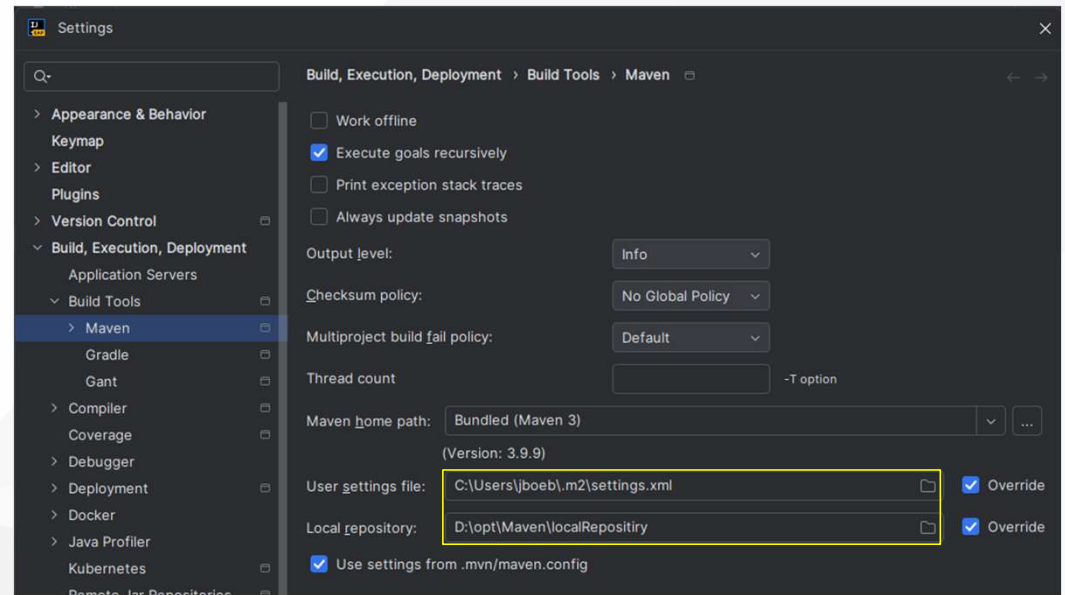


CONFIGURATION DE MAVEN DANS INTELLIJ

CONFIGURATION

Pour configurer Maven dans IntelliJ :

- File > Settings
 - Build, Exécution, Deployment
 - Build Tools
 - Maven





LES AUTRES OUTILS DE BUILD

LES AUTRES OUTILS DE BUILD

Il existe d'autres outils de build qui ont tous leurs avantages et leurs inconvénients.

Maven restant le plus utilisé aujourd'hui.

Source :

<https://linsolas.developpez.com/articles/java/outils/builds/>

- **Apache Ant**
 - opensource, intégré dans les IDE, CI/CD, intuitif et flexible
 - <https://ant.apache.org/>
- **Apache Ivy**
 - extension de Ant, moins verbeux que Maven mais faible popularité.
 - <https://ant.apache.org/ivy/index.html>
- **Gradle**
 - Plutôt récent (2008), script en groovy, peut réutiliser une infrastructure basée sur Maven. **Vrai concurrent à Maven**
 - <https://gradle.org/>



CONVERTIR UN PROJET "CLASSIQUE" EN PROJET "MAVEN" DANS INTELLIJ

GUIDE

Voici le **guide pas à pas** pour **convertir un projet Java “classique” en projet Maven** dans **IntelliJ IDEA** :

-  **Étape 1 : Ouvre ton projet existant**
 - Lance **IntelliJ IDEA**.
 - Ouvre ton projet Java (celui que tu as créé sans Maven).
-  **Étape 2 : Crée la structure Maven**
 - **via IntelliJ (recommandée)**
 - Dans la barre de menus :
File → New → Project from Existing Sources...
 - Sélectionne le dossier de ton projet.
 - Dans la fenêtre suivante, choisis **“Import project from external model” → Maven**.
 - Clique sur **Next**, puis **Finish**.
 - IntelliJ va créer un fichier pom.xml à la racine de ton projet et réorganiser les répertoires.

GUIDE

Maintenant que la structure est prête...finir le travail.

- **Étape 3 : injecter les dépendances utilisées dans le projet "classique" dans le pom.xml**

-  Maven ne détecte pas automatiquement les .jar externes (comme junit.jar, mysql-connector.jar, etc.) ajoutés manuellement dans le projet.
-  Maven ne les ajoute pas non plus dans le pom.xml.

- **Comment corriger ça**

- Si on a ajouté JUnit manuellement (par exemple ajouté au classpath via IntelliJ), il suffit de **l'ajouter à ton pom.xml**
- Puis clique sur **"Reload Maven Project"** (icône des deux flèches dans la fenêtre "Maven") → IntelliJ téléchargera automatiquement la dépendance depuis le dépôt central.

- **Étape 4 : Vérifie que tout compile**

- Dans IntelliJ, ouvre le terminal intégré et tape : **mvn compile**

MERCI !

Jérôme BOEBION
Concepteur Développeur d'Applications
Version 2.0 - révision 2025

Afpa
se former, avancer

